天池新闻推荐

代码来源：https://github.com/LogicJake/tianchi-news-recommendation

基于此源码进行学习，并做一些修改。

- 天池新闻推荐
  - 数据处理
  - 多路召回
    - ItemCF模型改进版
      - 相似度计算
        - 访问顺序
        - 时间间隔
        - 活跃用户行为影响（惩罚机制）
        - 热点数据影响（归一化）
      - 召回
        - 选取最近的K个物品
      - 召回指标
    - ItemCF模型
    - BiNetwork
      - 相似度计算
      - 召回
      - 召回指标
    - Word2Vec召回
      - 相似度计算
      - 召回
      - 召回指标
    - 双塔模型
    - 召回合并
      - MMS
      - 召回通道相似度
      - 最终召回指标的计算
  - 排序
    - 特征提取
    - 模型训练
      - 分离训练集和测试集

- LightGBM模型
- 训练

1. 数据处理

存在两种模式，online和offline，online直接使用训练数据和测试数据，offline中训练数据会进行切分，将一部分数据作为验证集，仅有一条用户信息的数据不会被采集为验证集。最后保存的数据是两个一个是 `df_query` 包括两列 `user_id` 和 `click_article_id` ，一个是 `df_click` 包括文件 `train_click_log.csv` 的全部列。

这部分涉及到一些数据处理的方法。

- `df.values.tolist()` ：将DataFrame转换为列表，每个元素是一个列表，列表中的元素是DataFrame中的一行数据。
- `df.groupby('user_id')['click_article_id'].apply(list)` ：将数据按照 `user_id` 进行分组，索引是 `user_id` ，值是该用户的所有 `click_article_id` 组成的列表。
- `df.head(1)` ：获取数据中的第一行数据。
- `df.tail(1)` ：获取数据中的最后一行数据。
- `df.shape` ：获取数据集的形状，返回一个元组，元组中第一个元素是行数，第二个元素是列数。
- `pd.concat([df.head(1), df.tail(1)])` ：将数据集的开头和结尾的数据进行拼接，返回拼接后的数据集。
- `pd.concat([df_train_click, df_test_click], sort=False).reset_index(drop=True)` ：拼接后重新设置索引。
- `pd.DataFrame(columns=df.columns)` ：创建一个空的DataFrame，列名与原始数据集相同。
- `pd.to_pickle(df, path)`：将DataFrame保存成pickle文件。
- `pd.read_pickle(path)` ：从pickle文件中读取数据，存储为DataFrame。

2. 多路召回

2.1 ItemCF模型改进版

2.1.1 相似度计算

`cal_sim(df)` 计算物品之间的相似度，基于同一个用户行为计算， `df` 是用户行为的数据。

单独抽取出用户列和物品列，通过 `group` 方法将同一个用户的行为进行聚合，聚合后的数据样式应该是这样子的

```
user_id | click_article_id
1       | [101, 102, 103]
2       | [201, 202]
```

访问顺序

原数据默认情况下是按照访问先后排序的，基于此引出一个正向顺序点击和反向顺序点击的概念，正向点击的权重是1，反向点击的权重是0.7。

例如数据存储在一个二维字典中 `sim_dict` 中，点击顺序是 A-B，那么在计算A对B的相似度 `sim_dict[A][B]` 的时候 `loc_alpha` 是1，在计算B对A的相似度时 `loc_alpha` 是0.7。

时间间隔

二者的相关性除开点击先后关系，还有间隔距离的关系，例如一个点击顺序是A-B-C，那么理论上来说，A对B的相似度应该比A对C的相似度要高。对于这个我们可以有以下解释：

- 即时相关性： 在同一会话或短时间内，用户的兴趣点非常集中。用户点击A后紧接着点击B，而非C，这强烈暗示了B与A的即时相关性高于C与A。这种"共现"的强度随着时间间隔的拉长而减弱。
- 点击顺序同时也是代表时间顺序，用户的兴趣会随时间变化。长时间间隔的两个点击，可能源于用户不同的兴趣状态。例如，白天点击"科研论文"，晚上点击"电影推荐"，这两者的关联性远低于短时间内连续点击的多篇"科研论文"。

为了量化这种"间隔距离"的影响，最有效的方法是在相似度计算中引入一个衰减系数。这个函数会给时间间隔短的行为对赋予更高的权重。

权重计算公式为：

```
loc_weight = loc_alpha * (0.9**(np.abs(loc2 - loc1) - 1))
```

活跃用户行为影响（惩罚机制）

例如有两个用户，一个是轻度用户，行为比较专注只点击了自己比较感兴趣的信息，另一个是重度用户，行为比较活跃，点击了多种信息，那么基于用户点击的相似度计算，就会有差异。

因此我们要进消除活跃用户对相似度的影响，通过对用户的点击序列长度取对数 `math.log(1 + len(items))` , 那么除以这个值，就可以消除用户点击序列长度对相似度的影响。但是这和 ***Swing模型*** 还是有区别的

热点数据影响（归一化）

热点数据对相似度影响较大，例如某个物品被点击的次数很多，我们在处理过程中是对两个物品之间的相似度不断做累加，

A被点击1000次，B被点击500次，C被点击8次，如果没有归一化，A-B的共现次数可能很高，那么其相似度的值就会很高，但是实际上B和C之间的相似度更高。故而会利用余弦相似度对其进行归一化。

我们是基于用户的点击行为进行相似度计算的，但是不同用户之间的行为方式不一样

```
sim_dict[item][relate_item] = value / math.sqrt(item_cnt[item] * item_cnt[relate_item])
```

最后得到的 `sim_dict` 就是相似矩阵，一起返回的还有用户-物品倒排表 `user_item_dict` ，key为用户ID，value为该用户点击过的物品列表。相似矩阵保存成 `pickle` 文件，以便后续过程随时取用。

2.1.2 召回

`recall(df_query, item_sim, user_item_dict, worker_id)` 用于计算召回结果， `df_query` 是查询数据， `item_sim` 是物品相似矩阵， `user_item_dict` 是用户物品倒排表， `worker_id` 是当前进程ID。

选取最近的K个物品

首先遍历 `df_query` ，有两列数据，分别是 `user_id` 和 `click_article_id` ，在offline模式下，其由两部分组成，一部分是valid，这一部分的 `click_article_id` 是用户点击过的物品列表，一部分是test，这一部分的 `click_article_id` 是-1。

依据最近点击过的2个物品，每个物品获取最相似的200个物品的相似度，依据顺序进行加权和累加。value为两个物品的相似度度，loc为最近交互顺序（0表示最近点击过的物品，1表示次近点击物品）。

```
rank[relate_item] += value * (0.7**loc)
```

在这里就不需要做归一化处理，因为如果多次出现，那么说明读者对这个物品有更高的兴趣，那么其值就会比较高，更让容易被推荐。最后选取 `rank` 数组中的前100物品，组成候选列表 `item_ids` ，作为该用户的召回结果。

新建一个 `DataFrame` ，分别有四列，分别是 `user_id` 、 `article_id` 、 `sim_score` 、 `label` 。这里的 `article_id` 是召回列表中的物品ID， `sim_score` 是物品的相似度，所以存储的都不是单个值，而是一个列表。 `label` 用于表示验证集中的物品是否存在于召回列表中，1表示存在，0表示不存在，对于测试集的数据， `label` 的值为Nan。最后将当前用户的召回信息存储到 `data_list` 中，最后将召回信息保存至 `pickle` 文件中。

2.1.3 召回指标

由于召回的过程是通过多线程计算的，所以需要对召回结果进行合并，合并后才能计算召回指标。

计算召回指标必须旨在offline模式下，因为存在验证集我们可以和实际的结果进行对比。首先根据 `article_id` 是否是 -1 筛选出验证集的数据，随后这里调用 `ultils.evaluate` 函数进行计算，第一个参数 `df` 是**验证集的召回结果**，第二参数 `total` 是 **验证集的用户数量** 。

函数中遍历 `df` 中，通过判断 `label` 是否为1，来判断该物品是否存在于召回列表中，如果存在就查找该物品在召回列表中的索引位置，然后计算其在召回列表的位置，随后计算5个不同推荐列表长度(5,10,20,40,50)下的命中率和MRR，以及在验证集中的整体命中率，输出的就是召回指标。

K	5	10	20	40	50
Hit Rate	0.3333	0.4541	0.5663	0.6589	0.6850
MRR	0.1966	0.2127	0.2205	0.2238	0.2244

Accuracy: 0.7557

2.2 ItemCF模型

这里原作者提到了使用Embedding向量计算相似度的方法并不好用，原因是实际上的物品数量特别多总共有364047个，而我们用于训练的数据涉及到的数量只有31116个，整整相差了十倍。存在一下的问题：

- **覆盖率不足**：由于训练数据仅覆盖了不到10%的物品，大量物品没有对应的Embedding向量或者向量质量不高，导致无法准确计算这些物品间的相似度。
- **冷启动问题严重**：对于新物品或很少被点击的物品，很难获得高质量的Embedding表示，使得基于内容的推荐效果不佳。
- **相似度计算偏差**：基于Embedding计算的相似度可能不能真实反映用户行为上的相似性，特别是在当前情况下，行为相似性比内容相似性更能体现用户偏好。
- **计算资源浪费**：为大量低频物品计算和存储Embedding向量消耗了大量的计算资源和存储空间，但这些向量在推荐中的作用却很有限，这里使用CPU进行处理的话需要一天，后来我修改为使用CPU并行处理也要很久的时间。最后使用pytorch调用GPU花了五分钟左右计算出了相似度矩阵。

此外，基于Embedding的方法忽略了用户行为序列中的重要信息，比如点击顺序和时间因素，这些都是在传统ItemCF中已经被证明有效的因素。虽然Embedding能够捕捉到一些语义层面的相似性，但在实际应用中，特别是在新闻推荐这类时效性强、用户兴趣变化快的场景下，行为数据往往比内容特征更加可靠。

这个是基于这个模型召回的指标

K	5	10	20	40	50
Hit Rate	0.0077	0.0104	0.0134	0.0183	0.0197
MRR	0.0045	0.0048	0.0051	0.0052	0.0053

Accuracy: 0.0293

从召回指标可以看出，基于Embedding的ItemCF模型表现远远不如传统的基于用户行为的ItemCF模型。命中率(Hit Rate)和平均倒数排名(MRR)都非常低，命中率仅为约2%，说明该方法确实不适合当前的数据分布和业务场景。

2.3 BiNetwork

2.3.1 相似度计算

这里的函数 `cal_sim(df)` 传入的是用户行为数据转化为 `DataFrame` ，首先基于用户进行分组得到 用户 - 物品列表，其次基于物品进行分组得到 物品 - 用户列表，后续的工作基于二者展开。

对 物品-用户列表进行遍历，基于当前物品找到所有点击过该物品的用户，然后基于这些用户找到其他点击过该物品的其他物品，最后计算这两个物品之间的相似度，具体实现如下：

```
sim_dict[item][relate_item] += 1 / (math.log(len(users)+1) * math.log(tmp_len+1))
```

`item` 表示当前物品， `relate_item` 表示该物品与其他物品的相似度， `users` 表示点击过当前物品的用户列表， `tmp_len` 对应用户点击过的物品数量，这里本质上使用的是一个改进过的 Item-IUF 算法，通过对数减少热门物品和活跃用户的影响，增加长尾效应。

这里十分简单粗暴直接进行了累加，并没有实现整体的归一化，个人认为是因为在累加的过程中已经通过对数降低了热门物品和活跃用户的影响。

2.3.2 召回

这里的召回过程与ItemCf中的召回过程一致，只是内部累加的时候没有添加顺序关系的权重，因为这里计算相似度的时候也没用到这个。

```
rank[relate_item] += value
```

2.3.3 召回指标

相比于改进的ItemCF这里前召回列表前五的HitRate和MRR差距不大，但是后续逐渐低于上一种方法。

K	5	10	20	40	50
Hit Rate	0.3364	0.4114	0.4869	0.5682	0.5935
MRR	0.2024	0.2126	0.2179	0.2208	0.2213

Accuracy: 0.6672

2.4 Word2Vec召回

2.4.1 相似度计算

这里通过自定义的函数 `word2vec(df_, f1, f2, model_path)` `df_` 是当前的用户记录就是从df_click读取的， `f1` 是需要提取的 `user_id` 字段名， `f2` 是需要提取的 `click_article_id` 字段名， `model_path` 是模型保存的路径。

这里再相似度计算的过程中将所有的信息按照用户进行分组，确保其为str列表，那么一个用户的交互信息按顺序视作一个句子，所有用户组成的“句子”用于Word2Vec进行训练。

```
model = Word2Vec(
  sentences=sentences,vector_size=256,window=3,min_count=1,sg=1,
  hs=0,seed=seed,negative=5,workers=10,epochs=1)
```

这里解释一下这几个参数的意思：

- `sentences`：训练的句子列表。
- `vector_size`：词向量的维度。
- `window`：窗口大小，这里设置为3，即考虑当前词左右3个词。
- `min_count`：最小词频，这里设置为1，即所有词都参与训练。
- `sg`：训练算法，这里设置为1，即使用skip-gram算法通过当前词预测其上下文词，当设置为0 的时候是CBOW算法通过上下文词预测当前词。
- `hs`：是否使用 Hierarchical Softmax，这里设置为0，即不使用。
- `seed`：随机数种子，保证每次训练的模型结果一致。
- `negative`：负采样数量，这里设置为5，即使用5个负样本。
- `workers`：训练线程数，这里设置为10，即使用10个线程进行训练。
- `epochs`：训练轮数，这里设置为1，即只训练1轮。

2.4.2 召回

在召回阶段，系统基于训练好的Word2Vec模型生成的文章向量进行相似文章的检索。对于每个用户，获取其历史点击过的文章列表，并只保留最近一次点击的文章用于召回计算。使用Annoy构建的文章向

量索引，通过计算向量相似度找到与用户最近点击文章最相似的100篇文章；将欧式距离转换为相似度得分，公式为 `sim_score = 2 - distance` 。类似到二维中，两个长度为 1 向量的距离范围为 [0, 2]，距离越小相似度越高。

对相似文章按得分排序，去除用户已经点击过的文章，保留为召回结果。如果待预测文章ID为-1则标记为缺失值，否则标记为0，并将实际点击文章标记为1。将结果保存为pickle文件供后续排序阶段使用。

2.4.3 召回指标

这里的召回指标相比于之间改进的itemCF和BiNetwork有很明显差距。

指标	5	10	20	40	50
Hit Rate	0.1475	0.2185	0.2864	0.3612	0.3825
MRR	0.0774	0.0869	0.0916	0.0943	0.0948

Accuracy: 0.4661

- 模型需要足够的用户行为序列才能训练出高质量的物品向量。但当前用户行为数据量可能不足，导致低频物品的向量表示不够准确，从而影响召回效果。
- 基于向量的相似度不一定能准确反映用户的实际兴趣。即使某些物品在向量空间中相似度高，但它们可能较为冷门，而用户在实际中更倾向于点击或交互热门物品。
- 模型未能充分利用用户行为序列中的顺序信息和时间信息，这些信息恰恰能捕捉用户的动态兴趣变化。

修改成依据最近两个，反而有所下降了。因为引入了新的信息会稀释用户的主要兴趣权重

指标\K值	5	10	20	40	50
Hit Rate	0.1002	0.1502	0.2097	0.2764	0.2997
MRR	0.0523	0.0590	0.0631	0.0654	0.0660

Accuracy: 0.3847

2.5 双塔模型

2.6 召回合并

原先的召回结果都已经保存成 `pickle` 文件，这里直接通过名字后缀读取即可。

2.6.1 MMS

读取之后基于每一种召回的 `user_id` 完成MMS（最小 - 最大标准化处理）。

之所以这样子做，是因为不同用户的行为模式是是不同的，重度活跃、兴趣广泛、行为噪声大的用户其相似度整体就是会偏高，新用户、兴趣专注、兴趣小众的用户相似度就会偏低。按用户做MMS可以消除这种用户间的分布差异，让每个用户的最佳候选都有可比性。另外将所有相似度的取值范围控制在围缩小到[0,1]，便于后续合并。

完成MMS之后对不同的召回结果进行加权求和，得到最终的推荐结果。

```
weights = {'itemcf': 1, 'binetwork': 1, 'w2v': 0.1}
```

我们前边分析过itemcf和binetwork整体效果比较好，但是w2v的效果和前两者相比还是有差距的，所以这里将其权重调整为0.1，其他两个为1。

2.6.2 召回通道相似度

这里通过 `recall_result_sim` 方法计算两个召回通道之间的相似度。对于 `d1` 和 `d2` 两个数据框，基于 `d1` 中的每一个用户找到召回列表，并进行长度的累加视作 `cnt` ，同时在 `d2` 中找到相同用户的召回列表，取二者的并集的长度累加视作 `hit_cnt` ，故而通过计算 `hit_cnt/cnt` 得到两个召回通道之间的相似度。

这是三种方法的相似度计算结果：

召回通道	itemcf	binetwork	w2v
itemcf	—	0.4246	0.2413
binetwork	0.4289	—	0.1138
w2v	0.2386	0.1113	—

事实上 itemcf-binetwork 和 binetwork-itemcf 的相似度值不完全一样，虽然其重合部分是一样的，但是他们的基准不一样就会导致最后的值不同。

2.6.3 最终召回指标的计算

相似度经过加权的信息都存储到了 `recall_final` 中去了，我们需要对同个用户下的召回的同一种物品相似度信息进行合并，在这里也就是求和操作。建立唯一的**用户-物品对**，与上面的求和后的相似进行合并。具体实现就是:

```
recall_score = recall_final[['user_id', 'article_id','sim_score']].groupby(['user_id', 'article_
recall_final = recall_final[['user_id', 'article_id', 'label'
                              ]].drop_duplicates(['user_id', 'article_id'])
recall_final = recall_final.merge(recall_score, how='left')
```

随后和先前的操作一样对验证集和测试集进行操作，方便计算召回指标。

指标\K值	5	10	20	40	50
Hit Rate	0.3720	0.4850	0.5787	0.6540	0.6766
MRR	0.2216	0.2368	0.2434	0.2461	0.2466

Accuracy: 0.8156

虽然整体的召回数量是之前单个召回通道的多倍，但是我们看前面的HitRate和MRR还是有大幅提升的。最后将召回结果保存为pickle文件供后续排序阶段使用。

3. 排序

3.1 特征提取

读取先前的合并的召回列表，对应的变量为 `df_feature` ，原先你总共有四列 `user_id` `article_id` `label` `sim_score`

文章信息

读取 `article.csv` 将所有的时间戳从毫秒级更换为秒级， 并且将这里的所有信息和原有的 `df_feature` 进行合并，得到新的 `df_feature` ，新增列为 `article_id` `category_id` `created_at_ts` `words_count`

小时信息

- `click_datetime_hour` ： 用于提取用户的小时信息，便于未来在不同时段对用户进行个性化推荐。

点击新闻的词数统计值

- `user_clicked_article_words_count_mean` ： 交互物品的平均词数
- `user_click_last_article_words_count` ： 最近交互的一条物品的词数

用户点击的相邻物品被创建的时间差

如果是记录中的第一条物品的间隔，那么设置为无穷大，其余的都计算与上一次的时间差，随后计算每个用户的平均时间差 `user_id_click_article_created_at_ts_diff_mean` 。

- `user_id_click_article_created_at_ts_diff_mean`：均值为正说明用户倾向于越看越新。比如在追更，或者按照时间线顺序刷Feed流；均值为负说明用户倾向于考古/挖坟。他在顺藤摸瓜挖掘以前的老内容；均值接近0说明用户阅读的文章发布时间很集中（一直看同一天的），或者他在新旧内容之间反复横跳（抵消了）。

用户点击的相邻物品的时间差

如果是记录中的第一条物品的间隔，那么设置为无穷大，其余的都计算与上一次的时间差，随后计算每个用户的平均时间差 `user_id_click_diff_mean` 。

- `user_id_click_diff_mean`：反映用户的活跃程度和浏览习惯。时间小我们可以认为是活跃的，时间大我们可以认为是不活跃的。但也可以认为是快节奏和慢节奏用户，那么对快节奏用户我们可以推荐短平快的内容，对慢节奏用户我们可以推荐比较有深度的内容。

点击和创建时间的平均值和标准差

将用户的点击和创建时间进行平均值和标准差计算，得到 `user_id_click_article_created_at_ts_mean/std` 和 `user_id_click_diff_mean/std` 。

- `user_id_click_article_created_at_ts_mean/std`：反应了用户对内容更新的偏好，时间小说明用户喜欢关注实时内容，时间大说明用户喜欢关注历史内容。这里相较于之前的引入了标准差的概念，是因为特征诊值会受异常值的影响，比如用户点击时间间隔特别长，那么该特征值就会特别高，那么我们就需要引入标准差来控制异常值的影响。

创建时间

- `user_click_last_article_created_time`：用户历史记录中最后一次点击的创建时间。捕捉用户的“当前兴趣上下文”，这个时间很旧，说明用户此时此刻正在“考古”或查找资料（例如，他刚点了一篇2015年的教程）。此时模型可能会倾向于推荐相关联的旧内容。这个时间很新，说明用户此时此刻正在追热点
- `user_clicked_article_created_time_max`：用户历史点击过的所有文章中，**发布时间最晚（最新）**的那篇文章的时间，判断用户对新内容是否有偏好。配合上最近点击的创建时间戳就可以得到一个比较综合的用户画像。

点击时间

- `user_click_last_article_click_time`：用户历史记录中最后一次点击的时间。捕捉用户的“当前兴趣上下文”，这个时间很旧，说明用户此时此刻正在“考古”或查找资料（例如，他刚点了一篇2015年的教程）。此时模型可能会倾向于推荐相关联的旧内容。这个时间很新，说明用户此时此刻正在追热点。
- `user_clicked_article_click_time_mean`：用户活跃时间的均值，感觉这个值比较奇怪，没什么意义，但是均值比较大或接近现在的可以认为是早期用户，均值较小的可以认为是新用户。但是比较

难去确定用户的活跃与否和新旧，一个活跃的早期用户可能该值比一个不活跃的新用户要小。只能说这是一个**融合特征**。

- `user_last_click_created_at_ts_diff` :召回物品创建时间与该用户最后一次点击物品创建时间的差值。
- `user_last_click_timestamp_diff` :召回物品创建时间与该用户最后一次点击物品点击时间的差值。
- `user_last_click_words_count_diff` :召回物品词数与该用户最后一次点击物品词数的差值。

计数统计

这里都是从历史记录中提取出来的

- `user_id_cnt` ： 用户点击物品的次数，协助判断用户的活跃程度。
- `article_id_cnt` : 物品被点击的次数，判断物品的冷热程度。
- `user_id_category_id_cnt` : 用户点击某一类别的次数，判断用户的兴趣偏好。

itemcf 特征提取

加载 `itemcf_sim.pkl` ，将物品之间的相似度加载进来，得到 `itemcf_sim` ，对于 `df_feature` 中的每一个 （`'user_id', 'article_id'`） 对进行一些计算。

- `user_id_article_id_itemcf_sim` : 召回物品与用户历史记录的累计相似度分数（基于位置和顺序关系）。
- `user_last_click_article_itemcf_sim` : 召回物品与用户最近交互物品的相似度

binetwork 特征提取

加载 `binetwork_sim.pkl` ，将物品之间的相似度加载进来，得到 `binetwork_sim` ，对于 `df_feature` 中的每一个 （`'user_id', 'article_id'`） 对进行一些计算。

- `user_last_click_article_binetwork_sim` : 召回物品与用户最近交互物品的相似度

w2v 特征提取

加载 `article_w2v.pkl` ，将物品之间的相似度加载进来，得到 `article_vec_map` ，对于 `df_feature` 中的每一个 （`'user_id', 'article_id'`） 对进行一些计算。

- `user_last_click_article_w2v_sim` : 这个特征表示用户最后一次点击的文章与当前待推荐文章之间的 Word2Vec 相似度
- `user_click_article_w2w_sim_sum_2` : 这个特征表示用户最近两次点击的文章与当前待推荐文章的 Word2Vec 相似度下相加

3.2 模型训练

3.2.1 分离训练集和测试集

读取保存后的特征矩阵 `df_feature` ， 按照是否存在标签分为训练集 `df_train` 和 测试集 `df_test` 除去 `label` 和 `created_at_datetime` 总共有28列特征，将这

3.2.2 LightGBM模型

初步学习了了下LightGBM的四个技巧，这里是直接调用函数来分析一下几个参数。

- `num_leaves`：树的最大叶子数
- `max_depth`：树的最大深度
-
- `learning_rate`：学习率，控制每棵树对最终结果的贡献率，这里设置为0.05比较小但有利于收敛
- `n_estimators`：最大迭代次数，设置为10000，因为前面的学习率比较小，迭代次数可以适当调大一些
-
- `reg_alpha`：L1正则化项，促进特征稀疏性，防止过拟合
- `reg_lambda`：L2正则化项，平滑权重分布，防止过拟合
-
- `subsample`：构建每一颗树使用的样本比例
- `feature_fraction`：每次分裂时考虑的样本比例
-
- `random_state`：随机数种子，保证每次运行结果一致
- `importance_type`：这里设置为 `gain` ，表示按特征增益排序，默认是 `split` ，表示按特征分裂排序
- `metric`：评价指标，这里设置为None，表示不评价，只输出模型结果，这里可以设置为 `auc` ，表示输出AUC值
- `early_stopping_rounds`：当连续n个迭代没有提升训练集的预测性能时，停止训练，防止过拟合

3.2.3 训练

使用 `GroupKFold` 进行交叉验证，将数据集分为5组，训练五次，每次1组为验证集，其他4组为训练集。一个用户的信息都出现在同一组，不会出现一个用户的信息出现在多组中，避免因同一用户数据同时出现在训练和验证集导致的过拟合。

特征矩阵 `df_feature` 中有28列特征，这里将特征矩阵 `df_feature` 拆分为 `X` 和 `y` ， `X` 为特征矩阵， `y` 为标签矩阵。使用 `fit` 进行拟合输入特征和标签，评价指标采用AUC，输出模型结果。

选取最好的模型基于验证集特征进行预测，得到 `pred` ，添加到从验证集抽取 `user_id` 、 `article_id` 、 `label` 三列后的矩阵 `df_oof` 中去，每一轮的 `df_oof` 组成最终的 `oof` 。

利用最好的模型基于测试集特征进行预测，得到 `pred` ，并对值除以5累加，因为使用的是五折交叉验证法。

获得当前fold下特征的importance，添加到列表中去。

最终将当前的模型存储起来，随后进行下一个fold的训练。

五折交叉验证完成后，计算各个特征的importance的平均值，输出结果如下：

排名	特征名称	特征重要性
0	sim_score	697983.776763
1	user_last_click_timestamp_diff	211545.720904
2	user_last_click_article_itemcf_sim	133503.350457
3	user_clicked_article_itemcf_sim_sum	81804.977421
4	user_last_click_article_w2v_sim	74116.162670
5	article_id_cnt	64021.827963
6	user_last_click_created_at_ts_diff	50063.632097
7	user_last_click_article_binetwork_sim	34339.955720
8	user_click_article_w2w_sim_sum_2	19368.762226
9	user_click_timestamp_created_at_ts_diff_mean	17966.236513
10	article_id	17947.003739
11	user_id_category_id_cnt	16460.236301
12	created_at_ts	16425.823065
13	user_click_last_article_click_time	16317.115566
14	user_click_timestamp_created_at_ts_diff_std	15708.173364
15	words_count	15508.712050
16	user_click_datetime_hour_std	14791.470288
17	user_last_click_words_count_diff	14123.630110
18	user_clicked_article_words_count_mean	13689.522006

排名	特征名称	特征重要性
19	user_id_click_diff_mean	13373.455882
20	user_id_click_article_created_at_ts_diff_mean	13055.948135
21	user_click_last_article_words_count	12571.715007
22	user_id	11843.316291
23	user_clicked_article_click_time_mean	11165.559107
24	user_click_last_article_created_time	10181.326920
25	user_clicked_article_created_time_max	9306.783414
26	user_id_cnt	7425.170274
27	category_id	5876.554095

对训练集排序后的结果进行重新评估，结果如下：

指标\K值	5	10	20	40	50
Hit Rate	0.4400	0.5743	0.6900	0.7702	0.7861
MRR	0.2643	0.2824	0.2905	0.2934	0.2938

Accuracy: 0.8156

在这一过程中没有改变物品的总数，所以Accuracy还是和之前保持一致的，但是Hit Rate和MRR 与召回合并的时候相比发生了巨大的改变。

对 `prediction` 进行处理，对每个用户选取前五个最大的物品，如果用户待选的物品数量小于5，则在召回的所有物品中随机选取。